

軟體品質與測試

第 5 章：黑箱測試、等價分割與屬性基礎測試 (PBT)

授課教師：薛念林 教授

系統內部就像一個黑盒子，我們不探究它「如何做」，
而是嚴格驗證它在各種極端輸入下，究竟「做了什麼」。
錯誤永遠隱藏在角落——掌握邊界與等價，方能以最小成本擊中最多缺陷。

本章重點導讀 (Key Highlights)

🕒 輸入域劃分與經典方法

- 5.1 黑箱測試概念與 JUnit 5 斷言：
 - 黑箱架構、Glenford Myers 測試哲學、`assertEquals` vs. `assertSame`
- 5.2 邊界值分析 (Boundary Value Analysis, BVA)：
 - 「錯誤隱藏在角落」、單一錯誤假設、獨立 ($4n+1/6n+1$) 與最壞情況
- 5.3 等價類分割測試 (Equivalence Partitioning, EP)：
 - 有效/無效等價類、弱/強涵蓋、單一無效原則與錯誤遮蔽 (Masking)
- 5.4 全成對組合測試 (Pairwise / All-Pairs)：
 - 2-Way 交互作用 (NIST 研究)、正交表 (OATS) vs. Pairwise (PICT)
- 5.5 案例優化與分類樹方法 (CTM)：
 - 案例精實化四大策略、Daimler-Benz 分類樹與測試矩陣

🔺 邏輯、情境與前沿測試

- 5.6 因果圖法與決策表測試 (Cause-Effect & Decision Table)：
 - 布林邏輯網絡、5 大約束條件 (E, I, O, R, M)、CAR 表規則化簡
- 5.7 狀態轉換測試 (State Transition Testing)：
 - 狀態機模型、狀態覆蓋 vs. 轉移覆蓋 (0-switch)、Stack 狀態斷言
- 5.8 屬性基礎測試 (Property-Based Testing, PBT)：
 - 數學不變量規格、jqwik 隨機測資與極簡收縮 (Shrinking)
- 5.9 使用案例與情境測試 (Use Case Testing)：
 - 端對端黑箱、基本流 (Happy Path)、替代流與例外流測試矩陣
- 5.10 經驗基礎測試：錯誤猜測與探索性測試：
 - 缺陷分類與防禦清單、探索性測試 (ET)、任務段管理 (SBTM)

5.1 黑箱測試概念與 JUnit 5 斷言

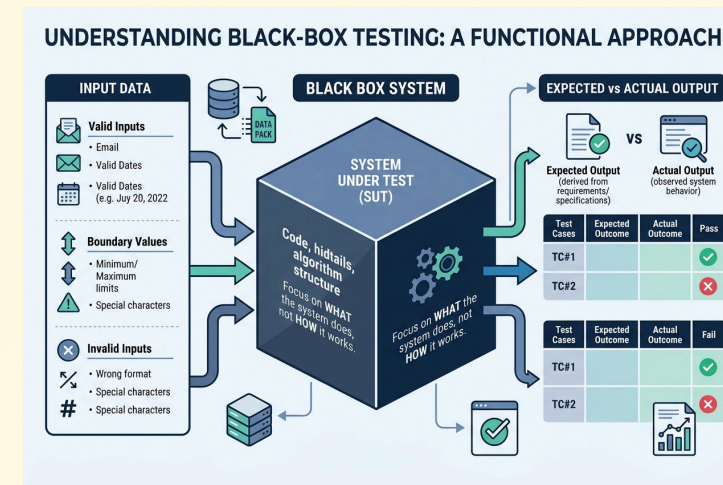
「不窺探程式碼內部結構，
純粹依據規格契約驗證系統外顯行為。」

黑箱測試 (Black-Box Testing) 概念架構

💡 黑箱測試（功能測試/規格測試）聚焦於系統「做什麼 (What)」，而非「如何做 (How)」。

黑箱測試三大核心要件

- 1. 測試輸入 (Input Data) :
 - 涵蓋合法資料 (Valid Inputs)、邊界極端值 (Boundary Values) 與格式錯誤的非法資料 (Invalid Inputs)。
- 2. 受測系統 (System Under Test, SUT) :
 - 內部原始碼與資料結構對測試人員隱藏（不透明黑盒），完全依據介面規格操作。
- 3. 輸出驗證 (Expected vs. Actual Output) :
 - 比對實際輸出與需求規格所定義之預期結果是否完全吻合。



軟體測試之父：Glenford J. Myers

💡 1979 年出版《The Art of Software Testing》，奠定黑箱測試、等價分割與邊界值分析的理論基石。

- 測試哲學的革命性定義：

- 「測試是為了發現錯誤而執行程式的過程，而不是為了證明程式沒有錯誤。」
- 破除「測試通過即系統完美」的迷思，將測試從被動驗收昇華為積極尋找缺陷的工程學。

- 黑箱測試三大經典方法奠基者：

- 率先系統化定義 等價類分割 (EP)、邊界值分析 (BVA) 與 錯誤猜測 (Error Guessing)。
- 主張「測試案例必須包含預期輸出」，否則測試員容易將錯誤輸出誤認為正常。

- 經典範例：三角形問題 (The Triangle Problem)：

- 提出聞名全球的「三邊長判斷三角形」測試題目，揭示最簡單的規格也常隱藏大量邊界與例外漏洞。



Glenford J. Myers (格倫福德·邁爾斯)

Author: The Art of Software Testing

JUnit 5 單元測試基礎與斷言機制

💡 Java 單元測試中最經典的踩坑陷阱：內容等價性與物件同一性的本質差別。

⚖️ assertEquals：內容值等價 (Value Equality)

- 運作原理：
 - 內部呼叫物件的 `a.equals(b)` 方法。
- 檢驗標的：
 - 兩個物件內部的「內容資料」是否相等。
- 適用情境：
 - 字串內容比對、自訂資料物件 (DTO)、數字運算結果驗證。

```
String s1 = new String("SQA");  
String s2 = new String("SQA");  
assertEquals(s1, s2); // ✅ 通過 (內容相同)
```

🎯 assertEquals：記憶體同一性 (Reference)

- 運作原理：
 - 內部比對記憶體指標位址 (即 `a == b`)。
- 檢驗標的：
 - 兩個參照變數是否指向 Heap 中的「同一記憶體實體」。
- 適用情境：
 - 單例模式 (Singleton)、快取池重用、工廠實體驗證。

```
assertEquals(s1, s2); // ❌ 失敗 (不同記憶體物件)  
String s3 = s1;  
assertEquals(s1, s3); // ✅ 通過 (指向同一位址)
```

概念核對問答 (CCQ 1)

概念核對問答 (CCQ 1)

問題情境：

【是非題】在 JUnit 單元測試中，`assertSame(a, b)` 斷言的作用與 `assertEquals(a, b)` 完全相同，兩者都是在驗證兩個物件的內容值是否相等（即比對 `a.equals(b)`）。

- A) 正確 (True)
- B) 錯誤 (False)



課堂互動

5.2 邊界值分析 (BVA)

「錯誤都隱藏在角落！

大多數的程式缺陷，都發生在邊界條件差之毫釐的瞬間。」

邊界值分析 (BVA) 核心哲學

💡 軟體工程界公認：輸入範圍的邊緣（極大值、極小值）比中間值更容易觸發 Off-by-one 缺陷。

為什麼錯誤隱藏在角落？

- 運算子誤用陷阱：
 - 程式設計師常在 `<` 與 `<=`、`>` 與 `>=` 之間犯錯。
- 程式碼良劣對照：

程式碼寫法	缺陷本質	測試結果
<pre>if (input <= 10)</pre>	無缺陷，邊界完整包含	input=10 通過
<pre>if (input < 10)</pre>	致命缺陷：10 被遺漏	input=10 測試失敗

- 結論：測試邊界點能以極高精準度曝光邊界比較缺陷。

單一錯誤假設 (Single Fault Assumption)

- 核心假設前提：
 - 在獨立變數系統中，絕大多數錯誤只需單一變數發生邊界異常即可彰顯，無需多個變數同時在邊界出錯。
- 取樣五大關鍵點：
 - 對於範圍 $[min, max]$ ，測試點取：
 - `min` (最小值)
 - `min+` (略大於最小值)
 - `norm` (區間內正常代表值)
 - `max-` (略小於最大值)
 - `max` (最大值)

獨立型邊界測試：4n + 1 公式

💡 針對 n 個彼此獨立的輸入變數，每次只讓 1 個變數處於邊界，其餘變數皆維持正常值 (norm)。

4n + 1 計算原理

- 每個變數貢獻 4 個邊界案例：
 - `min`, `min+`, `max-`, `max` (此時其他 $n - 1$ 個變數皆取 `norm`)。
- 加上 1 個全員正常中心點：
 - 所有 n 個變數皆取 `norm` 的基準案例。
- 案例總數公式：

$$\text{Total Test Cases} = 4n + 1$$
- 案例範例：若系統有 3 個獨立輸入變數，則測試案例數為 $4 \times 3 + 1 = 13$ 個。

▲ 三角形程式實戰案例 ($a, b, c \in [1, 200]$)

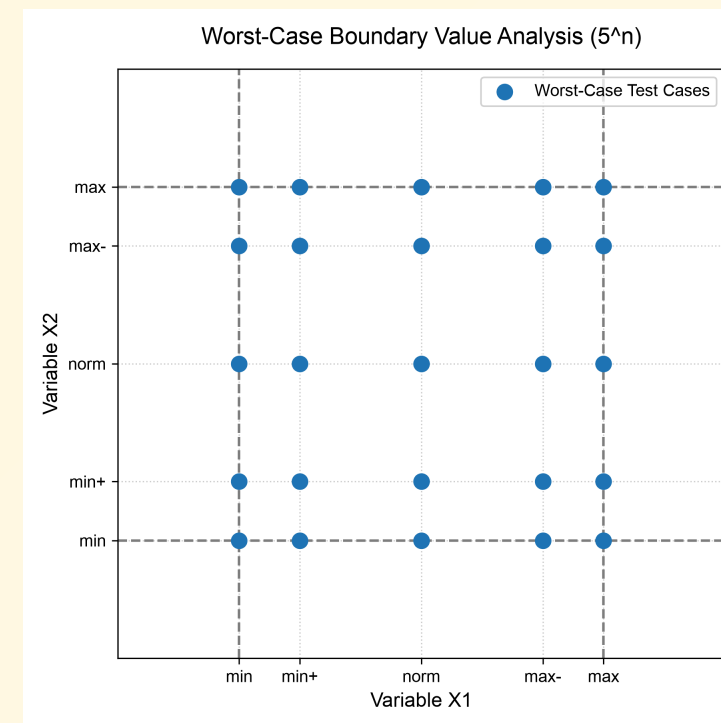
- 固定 $b = 100, c = 100$ ，變動 a ：
 - $(1, 100, 100), (2, 100, 100), (199, 100, 100), (200, 100, 100)$
- 固定 $a = 100, c = 100$ ，變動 b (4 個案例)
- 固定 $a = 100, b = 100$ ，變動 c (4 個案例)
- 全員正常基準案例：
 - $(100, 100, 100)$ 正三角形
- 總計剛好 13 個測試案例。

非獨立型邊界測試 (最壞情況邊界測試)

💡 當變數之間高度關聯（不符合單一錯誤假設）時，必須採用全乘積笛卡爾積進行最壞情況測試。

★ 乘積組合爆炸 (Combinatorial Explosion)

- 非獨立型一般邊界測試 (Worst-Case BVA) :
 - 每個變數取 5 個值 (`min`, `min+`, `norm`, `max-`, `max`)。
 - 總數為 5^n (若 $n = 3$, 需 $5^3 = 125$ 個案例；若 $n = 5$, 需 **3,125** 個)。
- 非獨立型強固邊界測試 (Worst-Case Robust BVA) :
 - 每個變數取 7 個值 (含 `min-`, `max+`)。
 - 總數為 7^n (若 $n = 3$, 需 $7^3 = 343$ 個；若 $n = 5$, 需 **16,807** 個案例！)。



邊界值測試方法全景總結

💡 四種邊界測試方法的案例數量與強度對比矩陣。

📊 測試案例數量矩陣

測試策略	單一錯誤假設 (獨立型)	多重錯誤假設 (非獨立/最壞情況)
標準邊界 (無越界)	$4n + 1$ (線性增長)	5^n (指數級增長)
強固邊界 (含越界)	$6n + 1$ (線性增長)	7^n (指數組合爆炸)

- 適用指標：
 - 數字型連續變數 (年齡、分數、金額、溫度)。
 - 對邏輯性識別碼 (如身分證字號、電話號碼) 較不適用。

🎯 工程選型建議

- 常規模組首選：
 - 獨立型強固邊界測試 ($6n + 1$)：在極低案例成本下，兼顧常態邊界與異常例外攔截。
- 高危關鍵金融/航太核心：
 - 最壞情況測試 (5^n)：只針對 2~3 個強關聯變數進行局部乘積，其餘變數維持獨立型，避免組合爆炸。

概念核對問答 (CCQ 2)

概念核對問答 (CCQ 2)

問題情境：

【單選題】假設某個受測方法接受 3 個彼此獨立的連續數值輸入參數。若採用「獨立型強固邊界測試 (Independent Robust BVA)」，其設計出的測試案例數量應為多少？

- A) 13 個
- B) 19 個
- C) 125 個
- D) 343 個



課堂互動

5.3 等價類分割測試 (EP)

「把無限的輸入空間劃分為有限的等價群組；
測試其中一個代表，就等於測試了整個群體。」

等價類分割測試 (Equivalence Partitioning)

💡 等價類的核心定義：同一個群組內的任意資料，若一個能找出 Bug，其他也必定能找出 Bug。

✚ 等價分割核心動機

- **達成完整測試**：確保每個業務分支與邏輯區間都有代表被測試到，避免盲區。
- **消除冗餘重複**：在同一個等價區間內測試 100 次只是浪費時間，挑選 1 個代表即可。
- **兩大等價類別**：
 - **有效等價類 (Valid Class)**：符合規格的合理合法輸入集合。
 - **無效等價類 (Invalid Class)**：超出範圍或格式異常的非法集合。

🎯 劃分維度與技巧

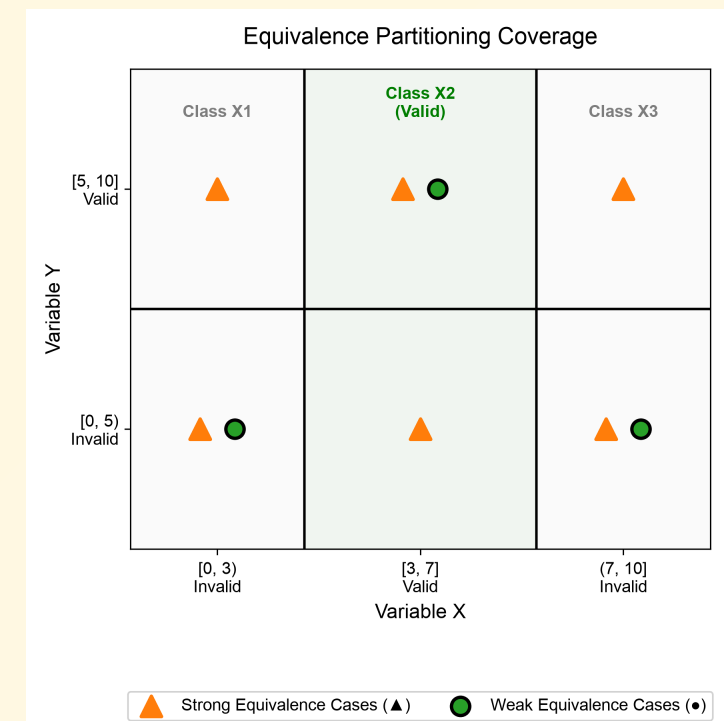
- **依輸入劃分**：
 - 數值範圍（如 $0 \leq score \leq 100 \rightarrow < 0, [0, 100], > 100$ ）。
 - 輸入個數（如「陣列為空」、「單一元素」、「多元素」）。
- **依輸出反推**：
 - 規格要求輸出 A, B, C, D $\rightarrow$ 倒推需要哪些輸入組合才能覆蓋這四種產出！

弱涵蓋 vs. 強涵蓋等價測試

💡 單一錯誤假設 vs. 多重錯誤假設在等價劃分中的延伸。

⚖️ 涵蓋強度深度比較

- 弱涵蓋測試 (Weak Coverage) :
 - 單一錯誤假設：要求每個變數的每個等價區間，在所有案例中至少出現過一次即可。
 - 測試案例數等於各變數分割數中的最大值 ($\max(m_i)$)。
- 強涵蓋測試 (Strong Coverage) :
 - 多重錯誤假設：要求測試所有變數等價區間的笛卡爾積組合（全部配對到）。
 - 測試案例數為所有變數分割數的乘積 ($\prod m_i$)。



Binary Search 實戰等價劃分案例

💡 二元搜尋的等價分割不僅看輸入 Key，更必須深入考量陣列長度與命中位置的等價關係。

📋 結構化等價劃分設計

- 陣列長度維度 (A) :
 - a^0 : 空陣列 (長度 0)
 - a^1 : 單一元素陣列
 - a^* : 多元素陣列 (可再細分奇數與偶數)
- 搜尋結果與位置維度 (F, C) :
 - f^t 找到 : 命中於頭部 (c^1)、中間 (c^m)、尾部 (c^l)
 - f^f 未找到 : 元素不存在於陣列中
- 輸入合法性維度 (K) :
 - k : 合法整數 ; $k!$: 非整數例外

🎯 弱涵蓋 8 大代表案例

- R_1 : 單元素，有找到 ($k = 7, A = [7]$)
- R_2 : 單元素，沒找到 ($k = 7, A = [8]$)
- R_3 : 多元素，命中頭部 ($k = 7, A = [7, 9, 11]$)
- R_4 : 多元素，命中中間 ($k = 9, A = [7, 9, 11]$)
- R_5 : 多元素，命中尾部 ($k = 11, A = [7, 9, 11]$)
- R_6 : 多元素，沒找到 ($k = 8, A = [7, 9, 11]$)
- R_7 : 空陣列，沒找到 ($k = 7, A = []]$)
- R_8 : 非法輸入，拋出例外 ($k = "abc"$)

領域規則導向等價分割：NextDate()

💡 單純考慮數值範圍是不夠的；優質的等價劃分必須融入業務規則（如大小月與閏年）。

✗ 膚淺的簡易分割（效能低）

- Month: $< 1, 1 \sim 12, > 12$ (3 類)
- Day: $< 1, 1 \sim 31, > 31$ (3 類)
- Year: $< 1812, 1812 \sim 2012, > 2012$ (3 類)
- 致命盲點：
 - 4/31 根本不存在！2/29 只有閏年合法！
 - 盲目劃分無法捕捉 2 月與大小月的核心跳日邏輯。

✓ 融入領域規則的專家分割

- 月份 (M):
 - M_1 : 30 天月 (4, 6, 9, 11)
 - M_2 : 31 天月 (1, 3, 5, 7, 8, 10, 12)
 - M_3 : 二月 (特殊處理)
- 日期 (D):
 - D_1 : $1 \sim 27$, D_2 : 28, D_3 : 29, D_4 : 30, D_5 : 31
- 年份 (Y):
 - Y_1 : 世紀平年 (1900), Y_2 : 閏年 (能被 4 整除), Y_3 : 一般平年

實務關鍵法則：單一無效原則與錯誤遮蔽 (Fault Masking)

💡 設計無效等價類測試案例時的鐵律：每次測試只允許包含一個無效等價類，其餘欄位必須全部合法！

🧐 為什麼禁止多個無效值同時輸入？

- 錯誤遮蔽效應 (Fault Masking Effect) :
 - 現代軟體通常採取「防禦性早退 (Early Exit)」或短路評估。
 - 當系統遇到第一個非法欄位時，往往立即中斷執行或拋出例外，不再檢查後續欄位。
- 後果：缺陷被遮蔽隱藏：
 - 若「密碼過短」與「信箱格式錯誤」同時輸入，程式在檢查密碼時拋錯中斷，後面的「信箱驗證 Bug」就永遠無法被觸發！

📌 有效類 vs. 無效類測試策略對比

- 有效等價類 (Valid Classes) —— 盡量合併：
 - 目標是驗證系統的正常運作路徑。
 - 每個測試案例應盡可能涵蓋多個尚未測試的有效等價類，以最小案例數獲得最高覆蓋。
- 無效等價類 (Invalid Classes) —— 嚴格隔離：
 - 目標是驗證各別錯誤防護與例外處理邏輯。
 - 每個測試案例只能包含一個無效等價類，其餘輸入條件一律設為已知合法值（單一缺陷原則）。

概念核對問答 (CCQ 3)

概念核對問答 (CCQ 3)

問題情境：

【單選題】在等價類分割測試 (Equivalence Partitioning) 中，「弱涵蓋 (Weak Coverage)」與「強涵蓋 (Strong Coverage)」分類法的主要區別是什麼？

- A) 弱涵蓋只測試有效等價類，而強涵蓋同時涵蓋有效與無效等價類
- B) 弱涵蓋基於單一錯誤假設（各等價類代表至少被測試一次），而強涵蓋基於多重錯誤假設（測試所有等價類的笛卡爾積全組合）
- C) 弱涵蓋不需要需求規格書，而強涵蓋必須完全依照 SRS 執行
- D) 弱涵蓋產生的測試案例數量一定比強涵蓋更多



課堂互動

5.4 全成對組合測試 (Pairwise)

「當組合爆炸讓窮盡測試成為不可能，
全成對測試用數學奇蹟，抓出 90% 的交互作用缺陷。」

組合爆炸難題與 2-Way 交互作用理論

💡 美國國家標準與技術研究院 (NIST) 研究：軟體系統中絕大多數缺陷皆由「單一變數」或「雙變數配對」所觸發。

💣 組合爆炸的殘酷現實

- 多參數系統的災難：
 - 假設系統有 10 個參數，每個參數各有 3 種可能值。
 - 全乘積窮盡測試需要：
 $3^{10} = 59,049$ 個測試案例！
 - 在工程實務上，根本沒有足夠的時間與預算執行窮盡測試。

🎯 NIST 實證經驗與成對假設

- 2-Way 交互作用理論：
 - 67% ~ 93% 的缺陷只要任意 2 個參數的交互作用就會觸發。
 - 需要 3 個以上參數交互觸發的罕見缺陷不足 10%。
- Pairwise (All-Pairs) 原則：
 - 只要任意兩個參數的所有可能取值組合都至少出現過一次即可！案例數從幾萬個劇降至數十個！

游泳池收費系統：全成對實戰對照

💡 透過交錯法與成對匹配，案例數量從全乘積的 12 個直接砍半至 6 個，覆蓋率絲毫不減。

📦 參數維度定義

- 日期 (d , 2 種) : d_1 假日, d_2 平日
- 時間 (t , 3 種) : t_1 清晨, t_2 中午, t_3 其他
- 身分 (m , 2 種) : m_1 會員, m_2 非會員
- 全乘積需求 : $2 \times 3 \times 2 = 12$ 個案例。

✅ 全成對測試套件 (6 個案例)

案例	時間 (t)	星期 (d)	會員 (m)
1	t_1 (清晨)	d_1 (假日)	m_1 (會員)
2	t_1 (清晨)	d_2 (平日)	m_2 (非會員)
3	t_2 (中午)	d_1 (假日)	m_1 (會員)
4	t_2 (中午)	d_2 (平日)	m_2 (非會員)
5	t_3 (其他)	d_1 (假日)	m_2 (非會員)
6	t_3 (其他)	d_2 (平日)	m_1 (會員)

(時間-星期、星期-會員、時間-會員兩兩組合 100% 涵蓋!)

全成對測試工具與放款計算案例

💡 求解最小 All-Pairs 測試集屬 NP-Hard 問題；現代工程皆採用高效率啟發式工具自動生成。

🔧 主流自動化工具

- PICT (微軟開源工具) :
 - 最流行的命令列工具，支援自訂約束條件 (Constraints) 與權重。
- ACTS (NIST 開源工具) :
 - Java 原生 JAR，支援高階 t -way 組合 (如 3-way, 4-way)。
- 放款利率計算案例 :
 - 利率受貸款金額、償還年限、青年貸款、會員身分影響，透過 PICT 瞬間將數百個全乘積案例壓縮至 10 餘個高價值案例。

房貸利率計算器

首頁 房貸資訊 聯絡我們

房貸利率計算器

貸款金額
10,000,000 元

償還年限
20 年

☒ 是否為會員 ☒ 是

☒ 是否為青年貸款 ☒ 是

計算利率

放款利率：1.85%

*本利率僅供參考，實際利率依銀行審核為準

概念核對問答 (CCQ 4)

概念核對問答 (CCQ 4)

問題情境：

【單選題】全成對測試 (Pairwise / All-Pairs Testing) 能夠大幅縮減測試案例數量，其在軟體工程上的核心理論依據是什麼？

- A) 軟體系統中的缺陷通常需要至少三個以上的參數交互作用才會觸發
- B) 絕大多數的軟體缺陷都是由「單一變數」或「任意兩個變數之間的交互作用 (2-way Interaction)」所引起的
- C) 配對測試是白箱測試的一種，可以直接涵蓋所有的程式碼分支路徑
- D) 成對測試可以保證 100% 涵蓋多變數系統的所有笛卡爾積組合



課堂互動

5.5 案例優化與 5.6 決策表測試

「條件錯綜複雜、規則千頭萬緒？
決策表化繁為簡，讓商業邏輯無所遁形。」

測試案例精實化四大策略

💡 面對數千個潛在案例，透過工程優化策略進行剪枝，保留最高殺傷力的測試子集。

✂️ 1. 分離例外與 2. 配對組合

- 分離等效例外：
 - 若所有因子的非法輸入皆導向相同的錯誤處理（如拋出 400 錯誤），無需測試所有因子的例外排列組合，只需將例外抽取為獨立集合測試。
- 2-Way 配對組合：
 - 將不相關的因子改以 Pairwise 方式結合，指數級案例瞬間降為多項式級。

🎯 3. 邊界鎖定與 4. FMEA 風險優先

- 鎖定邊界替代隨機：
 - 等價區間內部只挑 1 個代表，將精力聚焦於 $min, min+, max-, max$ 。
- FMEA 故障模式分析：
 - 高風險、核心金流、常變更模組採用強涵蓋；低風險輔助模組降級為弱涵蓋。

工業界視覺化等價組合：分類樹方法 (CTM)

💡 Classification Tree Method (CTM) 由 Daimler-Benz 研發，現為 ISO/IEC/IEEE 29119-4 認可的圖形化黑箱測試設計標準。

🌳 1. 分類樹架構 (Classification Tree)

- 分類 (Classifications) :
 - 將輸入域依據測試觀點分解為若干互斥維度 (例如：身分、支付方式、訂單金額)。
- 類別 (Classes) :
 - 每個分類維度下，劃分出互斥且完整的等價子集 (例如：VIP / 一般會員 / 訪客)。
- 視覺化階層分解 :
 - 支援子分類 (Sub-classifications)，可清晰呈現複雜前置依賴關係。

📊 2. 測試規格組合表格 (Combination Table)

- 下方規格矩陣 (Matrix) :
 - 表格的欄對應樹狀葉節點 (類別)，每一列代表一個具體的測試案例 (Test Case)。
- 打點勾選設計 :
 - 測試人員只需在每一列的各分類中勾選一個代表類別 (落實弱涵蓋、強涵蓋或 Pairwise)。
- 實務優勢：直觀易讀、自動防漏、極利於向利害關係人與稽核員審查測試完整性。

概念核對問答 (CCQ 5)

概念核對問答 (CCQ 5)

問題情境：

【單選題】關於「正交表測試 (Orthogonal Array Testing, OATS)」與一般「全成對測試 (Pairwise Testing)」的比較，下列敘述何者正確？

- A) 正交表測試是隨機產生的，而 Pairwise 必須透過數學嚴格推導
- B) 兩者都關注參數間的配對，但正交表更強調各因子組合的「均勻平衡性 (正交性)」，而 Pairwise 僅要求任意兩因子的組合至少出現一次，因此案例數量通常更少且更有彈性
- C) 只要變數的個數相同，正交表與 Pairwise 產生的測試案例清單必定完全一致
- D) 正交表測試只能處理二分值 (True/False) 的變數組合



課堂互動

5.6 因果圖與決策表測試

「從規格描述的邏輯網絡中，
理清所有原因、結果與制約條件，系統化導出精準決策。」

邏輯相依規格建模：因果圖法 (Cause-Effect Graphing)

💡 Glenford Myers 提出因果圖法：將自然語言規格轉譯為布林邏輯電路網絡，作為推導決策表的正規工具。

⚡ 核心構成要件

- 原因 (Causes, C_i) :
 - 系統的輸入條件或前置狀態 (取值 0 或 1)。
- 結果 (Effects, E_j) :
 - 系統產生的輸出動作或狀態改變 (取值 0 或 1)。
- 4 大基本布林邏輯關係 :
 - 恆等 (Identity) : 若 $C_1 = 1$ 則 $E_1 = 1$; 否則 $E_1 = 0$ 。
 - 非 (NOT, $\sim$) : 若 $C_1 = 1$ 則 $E_1 = 0$; 否則 $E_1 = 1$ 。
 - 或 (OR, $\vee$) : 若任一 $C_i = 1$ 則 $E = 1$ 。
 - 與 (AND, $\wedge$) : 若所有 $C_i = 1$ 則 $E = 1$ 。

🎯 因果圖 5 大約束條件 (Constraints)

- 輸入端約束 (Causes Constraints) :
 - E (Exclusive 互斥) : 最多只有一個為 1 ($C_1 + C_2 \leq 1$)。
 - I (Inclusive 包含) : 至少有一個為 1 ($C_1 + C_2 \geq 1$)。
 - O (One and only one 唯一) : 恰好只有一個為 1 ($C_1 + C_2 = 1$)。
 - R (Requires 要求) : 若 $C_1 = 1$, 則 C_2 必須為 1 ($C_1 \implies C_2$)。
- 輸出端約束 (Effects Constraints) :
 - M (Masks 遮蔽) : 若 $E_1 = 1$, 則 E_2 強制被抑制為 0。

從因果圖到決策表 (CAR) 的轉換流程

💡 因果圖是橋樑，決策表是成果：四步驟消除規格盲區與邏輯矛盾。

🔄 系統化 4 步轉換程序

- Step 1：拆解因果因子：
 - 分析規格，為每個輸入條件與輸出動作編號命名 ($C_1..C_n, E_1..E_m$)。
- Step 2：繪製邏輯網絡與標註約束：
 - 用邏輯閘連接原因與結果，加上 E, I, O, R, M 約束。
- Step 3：由結果倒推條件組合：
 - 針對每個輸出 $E_j = 1$ 的狀態，倒推需要哪些輸入組合；排除違反約束的非法組合。
- Step 4：生成並化簡決策表 (CAR)：
 - 將合法因果組合填入決策表作為規則 (Rules)，合併等效條件項。

💎 因果圖法核心價值

- 打破自然語言模糊性：
 - 規格書中常見的「如果...而且...除非...否則...」常常語意含混，因果圖能第一時間揪出規格矛盾或遺漏。
- 防止無效組合爆炸：
 - 藉由互斥 (E) 與唯一 (O) 約束，在初期就剪除數十種不可能發生的輸入組合，大幅減輕後續測試成本。
- 全自動化生成潛力：
 - 現代 SAT/SMT Solver 可直接將因果圖轉換為最簡可滿足解與測試案例。

決策表測試 (Decision Table Testing) 架構

💡 針對高相依性、高制約的複雜商業邏輯，透過 Condition-Action-Rule (CAR) 進行邏輯收斂。

📋 CAR 決策表四大區塊

- 條件樁 (Condition Stubs) :
 - 列出影響決策的所有輸入條件與前置狀態 (如 C_1, C_2, C_3)。
- 動作樁 (Action Stubs) :
 - 列出系統可能觸發的所有輸出動作 (如 A_1, A_2)。
- 條件項 (Condition Entries) :
 - 填入各條件之取值 (Y, N 或 不在乎 $-$)。
- 動作項 (Action Entries) :
 - 標記該規則下應執行的動作 (X)。

✂ 規則展開與化簡 (Simplification)

- 初始全規則展開 :
 - k 個布林條件共有 2^k 條初始規則。
- 不相關化簡 (Don't Care) :
 - 若兩條規則僅有一項條件不同，但觸發相同動作，該條件可化簡為 $-$ (不在乎) 並合併為單一規則！
- 效益：杜絕規格矛盾與遺漏，需求分析與測試共用。

三角形判斷決策表 (CAR Table) 實戰

💡 三角形判斷邏輯化簡後的 CAR 決策表，精確對應 6 條測試規則。

條件與動作	R_1	R_2	R_3	R_4	R_5	R_6
條件 (Conditions)						
C_1 : 滿足三角不等式? ($a < b + c \wedge b < a + c \wedge c < a + b$)	N	Y	Y	Y	Y	Y
C_2 : $a = b$?	-	Y	Y	N	N	N
C_3 : $b = c$?	-	Y	N	Y	N	N
C_4 : $a = c$?	-	Y	N	N	Y	N
動作 (Actions)						
A_1 : 輸出「非三角形」	X					
A_2 : 輸出「正三角形」		X				
A_3 : 輸出「等腰三角形」			X	X	X	
A_4 : 輸出「不等邊三角形」						X

概念核對問答 (CCQ 6)

概念核對問答 (CCQ 6)

問題情境：

【單選題】在軟體測試實務中，下列哪一種受測情境最適合優先採用「決策表測試 (Decision Table Testing)」來設計案例？

- A) 系統輸入參數彼此完全獨立，且有連續性數值邊界
- B) 輸入參數之間存在複雜的商務邏輯與制約關係，不同的條件組合會觸發不同的系統動作或輸出結果
- C) 系統的輸出僅與目前輸入值有關，與輸入條件的組合邏輯無涉
- D) 系統的運作強烈依賴時間序列與物件歷史狀態的轉移



課堂互動

5.7 狀態轉換測試

「沒有回傳值的 void 方法怎麼測？
答案在於物件內部的狀態轉移。」

事件驅動與狀態機 (State Machine) 測試

💡 現代系統多為事件驅動 (Event-Driven)；測試重點在於：受到事件激發後，狀態轉換是否如預期反應。

✈️ 航空訂票狀態機生命週期

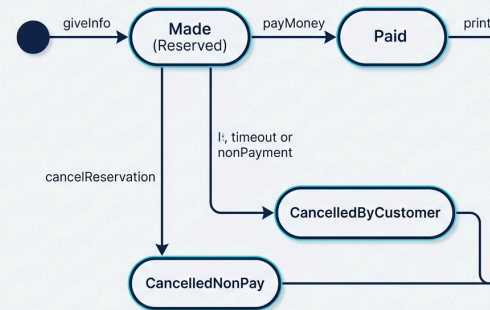
- 主流程 (Happy Path)：

- Made (已預約) $\xrightarrow{\text{payMoney}}$ Paid (已付款) $\xrightarrow{\text{print}}$ Ticketed (已開票) $\xrightarrow{\text{giveTicket}}$ Used (已使用)

- 取消與逾期例外分支：

- 在 Made/Paid/Ticketed 下觸發 `cancel` → 轉移至 `CancelledByCustomer`。
- 在 Made 下付款逾時 (`payTimeExpire`) → 轉移至 `CancelledNonPay`。

State Machine Diagram for Airline Ticket Booking in software engineering



狀態測試覆蓋準則與測試路徑設計

💡 狀態覆蓋 (Node) vs. 轉移覆蓋 (Edge)：轉移覆蓋強度遠高於狀態覆蓋。

🎯 覆蓋強度層次

- 狀態覆蓋 (State Coverage)：
 - 每個狀態節點至少被造訪一次。
- 轉移覆蓋 (Transition Coverage)：
 - 每個狀態轉移弧線至少被執行一次（必達狀態覆蓋！）。
- N-Switch 覆蓋：
 - 0-Switch（單次轉移）、1-Switch（連續 2 次轉移序列）。

🛤️ 5 條全轉移覆蓋測試序列

- 路徑 1：`giveInfo` → `payMoney` → `print` → `giveTicket`（正常使用）
- 路徑 2：`giveInfo` → `payMoney` → `print` → `cancel`（開票後取消）
- 路徑 3：`giveInfo` → `payMoney` → `cancel`（付款後取消）
- 路徑 4：`giveInfo` → `cancel`（預約後取消）
- 路徑 5：`giveInfo` → `payTimeExpire`（逾期末付）

物件狀態測試：Stack 實戰

💡 物件導向測試精髓：執行 void 動作（如 push/pop），斷言內部狀態變化（`isEmpty()`，`isFull()`）。

```
// 測試狀態轉移至 isFull
Stack s = new Stack(3);
s.push(100);
assertFalse(s.isFull()); // 狀態: NotEmpty
s.push(200);
s.push(300);
assertTrue(s.isFull()); // 狀態轉移成功: isFull

// 測試在 isFull 狀態下再次 push 應拋出例外 (強固性)
assertThrows(StackFullException.class, () -> s.push(400));
```

先設初始狀態，再激發事件：先 push 3 次建立滿狀態，再驗證越界拋錯。

回歸狀態驗證：連續 pop 3 次後，斷言 `assertTrue(s.isEmpty())`。

概念核對問答 (CCQ 7)

概念核對問答 (CCQ 7)

問題情境：

【單選題】在狀態測試 (State Testing) 中，關於「狀態覆蓋 (State Coverage)」與「轉移覆蓋 (Transition Coverage)」的強度關係，下列敘述何者正確？

- A) 達到狀態覆蓋必定代表同時達到了轉移覆蓋
- B) 轉移覆蓋的強度大於狀態覆蓋；若測試案例達到了轉移覆蓋（驗證了所有可能的轉移路徑弧線），則必定已涵蓋了所有狀態節點
- C) 兩者互相獨立，沒有任何包含或強弱關係
- D) 狀態測試不需要考慮無效轉移（即在某狀態下輸入非法事件的系統反應）



課堂互動

5.8 屬性基礎測試 (PBT)

「不再人工苦思特定測資，
而是宣告系統永不受協的數學不變量。」

範例測試 (EBT) vs. 屬性測試 (PBT) 典範轉移

💡 從 Example-Based 走向 Property-Based：讓框架自動產生萬組極端隨機測資，尋找致命反例。

傳統範例測試 (Example-Based)

- 手動設計特定案例：
 - 挑選輸入 2 與 3，斷言輸出必為 5。
- 致命瓶頸：
 - 覆蓋率高度取決於工程師的直覺與經驗；
 - 極容易遺漏包含極大值、空字串、Unicode 特殊字元之極端漏洞。

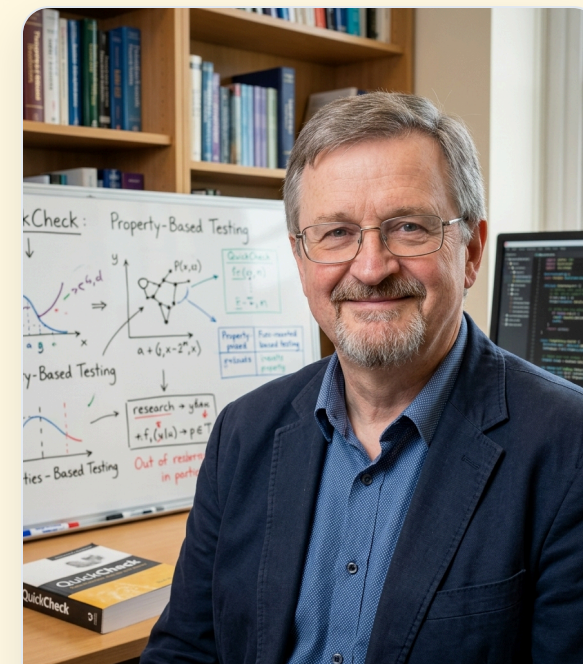
屬性基礎測試 (Property-Based)

- 定義通用不變量 (Invariant)：
 - 不管輸入何值，加法必滿足「交換律 $a + b = b + a$ 」。
- 三大核心機制：
 - 1. 屬性 (Property)：永恆成立之數學真理。
 - 2. 生成器 (Generator)：自動產生數萬組隨機邊界資料。
 - 3. 測資收縮 (Shrinking)：遇錯自動簡化為極簡反例！

屬性基礎測試 (PBT) 先驅：John Hughes

💡 2000 年發明 QuickCheck，開創「不要寫測試案例，定義程式屬性」的現代測試典範。

- QuickCheck 的誕生與榮譽：
 - 瑞典查爾摩斯工學院教授、ACM Fellow。
 - 2000 年與 Koen Claessen 共同發表 QuickCheck，獲頒 ACM SIGPLAN 最具影響力論文獎。
- 測試思維的重大躍遷 (Paradigm Shift)：
 - 提倡「Don't write tests, specify properties!」
 - 人類專注於找出系統應遵守的永恆數學不變量，將繁瑣的測資生成交給電腦。
- 革命性發明：測資收縮 (Shrinking)：
 - 解決了隨機測試「反例過大、無法除錯」的痛點，以演算法秒級自動二分縮減至最小致命測資。
- 現代 PBT 生態系基石：
 - 創立 Quviq 並在愛立信、Volvo 驗證；Java `jqwik`、Python `Hypothesis` 皆承襲其設計。



John Hughes (約翰·休斯)
QuickCheck Co-creator / ACM Fellow

jqwik 實戰與測資收縮 (Shrinking) 機制

💡 當一萬筆測資中長度為 100 的亂碼陣列出錯時，PBT 框架能自動縮減為 `[0, -1]` 幫助工程師秒級除錯。

```
import net.jqwik.api.*;

public class AdditionProperties {

    // 測試加法交換律：a + b 必須等於 b + a (自動執行 1000 次隨機測試)
    @Property
    void additionIsCommutative(@ForAll int a, @ForAll int b) {
        int result1 = a + b;
        int result2 = b + a;
        assert result1 == result2;
    }

    // 測試恆等律：任何整數加上 0 必須等於自己
    @Property
    void additionWithZeroHasNoEffect(@ForAll int a) {
        assert a + 0 == a;
    }
}
```

概念核對問答 (CCQ 8)

概念核對問答 (CCQ 8)

問題情境：

【是非題】在屬性基礎測試 (Property-Based Testing, PBT) 中，我們不需要手動為每一組測試寫出確切的預期輸出數值，而是定義程式執行時必須永遠維持的「屬性或不變量 (Invariants)」，並交由測試框架隨機生成大量測資來自動尋找反例。

- A) 正確 (True)
- B) 錯誤 (False)



課堂互動

5.9 使用案例與情境測試

「跳出單一欄位與單元函數的微觀視角，
從真實使用者的業務場景，驗證端對端（End-to-End）系統行為。」

系統層級黑箱核心：使用案例測試 (Use Case Testing)

💡 ISTQB 國際認證五大黑箱技術之一：基於需求規格中的 Use Case / User Story，驗證跨元件的端對端業務流。

三大關鍵業務流程路徑

- **基本流 (Basic Flow / Happy Path)：**
 - 系統在最理想、無任何錯誤狀況下，使用者順利達成目標的主成功流程。
- **替代流 (Alternative Flows)：**
 - 使用者達成相同目標的合法次要分支路徑（如：使用折價券付款、更換取件超商）。
- **例外流 (Exception Flows)：**
 - 發生錯誤或中斷導致目標無法達成的防禦路徑（如：卡片過期、庫存不足、網路斷線）。

覆蓋準則與測試設計原則

- **1. 最低覆蓋 (Minimum)：**
 - 必須至少設計 1 個測試案例完整執行「基本流 (Happy Path)」。
- **2. 完整業務覆蓋 (Full Coverage)：**
 - 每個替代流與每個例外流，均至少有 1 個測試案例走過。
- **3. 組合情境測試 (Scenario Testing)：**
 - 將多個替代流與例外流按真實用戶行為序列組合（例如：密碼錯一次後成功登入、取消又重新下單）。

使用案例測試矩陣實戰：ATM 提款流程

💡 以典型 ATM 提款使用案例，推導系統級端對端黑箱測試案例矩陣。

測試案例 ID	測試情境說明	流程路徑類型	輸入資料與前置條件	預期系統反應
TC-01	正常提款成功	基本流 (Happy Path)	帳戶餘額 5000，提款 1000，密碼正確	出鈔 1000、扣款成功、列印明細退卡
TC-02	提款前先查詢餘額	替代流 (Alternative)	插入晶片卡 → 查詢餘額 → 再提款 1000	顯示餘額無誤、順利出鈔退卡
TC-03	提款金額超出單次上限	例外流 (Exception 1)	提款輸入 30000（上限為 20000）	螢幕提示「超出單筆上限」、拒絕出鈔
TC-04	帳戶餘額不足	例外流 (Exception 2)	帳戶餘額 500，請求提款 1000	提示「可用餘額不足」、退卡
TC-05	連續輸錯密碼 3 次	例外流 (Exception 3)	故意連續輸入 3 次錯誤 PIN 碼	第三次錯誤時吞卡、通報行內風控防盜

5.10 經驗基礎測試：錯誤猜測與探索性測試

「規格書永遠寫不完所有現實的混亂。
頂尖測試者的直覺、歷史缺陷經驗與即時探索，是抓出致命漏洞的終極防線。」

經驗基礎測試 (Experience-based) 與錯誤猜測法

💡 Glenford Myers 黑箱三大基石之三：基於過往 Defect 統計、工程師易犯錯誤與防禦性思維的「直覺性攻擊」。

🎯 錯誤猜測法 (Error Guessing) 核心哲學

- 並非盲目碰運氣：
 - 它是根據**缺陷分類庫 (Defect Taxonomy)**、架構弱點與工程師盲區所進行的高效能「故障注入 (Fault Injection)」。
- 與規格基礎技術的共生關係：
 - 規格測試 (EP/BVA) 確保「功能如期運作」；錯誤猜測則主動尋找「規格未提及但實務必爆的死角」。

💣 實務經典攻擊清單 (Fault Injection Checklist)

- 空值與邊界：`null`、未初始化、空字串 `""`、全空格、超長字串 (Buffer Overflow)。
- 特殊字元與編碼：全形半形、Emoji、換行符 (`\r\n`)、SQL 跳脫符 (`' OR 1=1--`)、XSS (`<script>`)。
- 數值極端運算：除以零 (`1/0`)、整數溢位 (`Integer.MAX_VALUE + 1`)、浮點數精度誤差 (`0.1 + 0.2`)。
- 時間與併發：跨時區夏令時間、閏秒、快點兩下造成的重複扣款 (Race Condition)。

敏捷時代的利器：探索性測試 (Exploratory Testing)

💡 Cem Kaner & James Bach 發揚：測試學習 (Learning)、測試設計 (Design) 與測試執行 (Execution) 同步交織進行。

⚖️ 探索性測試 vs. 腳本化測試 (Scripted)

- 腳本化測試 (Scripted Testing) :
 - 事先撰寫精確步驟 → 盲目按步就班執行 → 適合自動化回歸測試，但易受「殺蟲劑悖論 (Pesticide Paradox)」鈍化。
- 探索性測試 (Exploratory Testing) :
 - 根據受測系統的最新即時反饋，靈活調整下一個測試策略，主動追擊可疑異狀，抓蟲效率極高。

🕒 基於任務段的測試管理 (SBTM 架構)

- 測試任務書 (Charter) :
 - 明確探索範圍與啟發點（探索什麼目標？使用什麼工具？尋找哪類弱點？）。
- 時間盒 (Time-box) :
 - 設定 60 ~ 90 分鐘專注衝刺，不被打擾，維持高敏銳度。
- 任務紀錄 (Session Log) :
 - 產出結構化紀錄：測試覆蓋軌跡、發現的 Bugs、未解疑點與後續測試建議。

附錄：課堂互動參考解答

💡 本章課堂互動 (CCQ 1 ~ CCQ 8) 官方標準解答與核心觀念解析總整理。

🚫 CCQ 1 ~ 4：斷言、邊界與組合

- **CCQ 1【錯誤 (B)】**：`assertEquals` 驗證內容相等 (`equals`)，`assertSame` 驗證記憶體位址同一性 (`==`)。
- **CCQ 2【19 個 (B)】**：3 變數獨立強固 BVA 案例數為 $6n + 1 = 6 \times 3 + 1 = 19$ 。
- **CCQ 3【單一 vs 笛卡爾積 (B)】**：弱等價基於單一錯誤假設，強等價要求所有變數分割之全乘積。
- **CCQ 4【2-Way 交互作用 (B)】**：NIST 實證絕大多數缺陷由單變數或雙變數交互引發，Pairwise 具極高性價比。

🎯 CCQ 5 ~ 8：正交、決策、狀態與屬性

- **CCQ 5【正交平衡性 vs 靈活度 (B)】**：正交表要求兩兩配對次數嚴格相等，Pairwise 僅求 ≥ 1 故更精實。
- **CCQ 6【複雜商務邏輯制約 (B)】**：決策表最適於多條件輸入與輸出動作具複雜因果關聯之場景。
- **CCQ 7【轉移覆蓋大於狀態覆蓋 (B)】**：轉移覆蓋走遍所有邊弧，必定已造訪所有狀態節點。
- **CCQ 8【正確 (A)】**：PBT 核心為規格即不變量，自動生成海量輸入並透過 Shrinking 回報極簡反例。